Raport z testów aplikacji „Fuchacz”

Autorzy: Mikołaj Kalejta, Jan Gasztold, Maciej Adamski

Marzec 2026

Spis treści

1 Wstęp **2**

2 Narzędzia testowe **2**
2.1 Backend 2
2.2 Frontend 2

3 Pipeline CI/CD **3**

4 Pokrycie kodu **4**
4.1 Backend 4
4.2 Frontend 6

5 Zasady pisania testów **6**

6 Podsumowanie liczby testów **7**

7 Rodzaje testów **9**
7.1 Frontend 9
7.1.1 Testy komponentów 9
7.1.2 Testy manualne 9
7.2 Backend 9
7.2.1 Testy integracyjne 9
7.2.2 Testy jednostkowe 9
7.2.3 Testy manualne 9

8 Wnioski **10**

1 Wstęp

Aplikacja **Fuchacz** jest mobilną platformą pośredniczącą w nawiązywaniu kontaktów pomiędzy pracownikami poszukującymi zatrudnienia a pracodawcami oferującymi pracę. Jej sercem jest mechanizm przeglądania ofert oparty na gestach (swipe), wzorowany na popularnych aplikacjach randkowych — kandydat akceptuje lub odrzuca ofertę jednym ruchem palca, a pracodawca przegląda nadesłane aplikacje i wybiera osoby, z którymi podpisuje umowę. Aplikacja obsługuje logowanie tradycyjne (e-mail i hasło) oraz logowanie przez konto Google, zarządzanie CV, filtry wyszukiwania, a także panel administracyjny.

Cel testowania

Głównym celem przeprowadzonych testów było **zbadanie poprawności zaimplementowanych funkcjonalności** oraz **sprawdzenie kompatybilności poszczególnych modułów** tworzących system. Testy miały potwierdzić, że komunikacja warstwy backendowej z bazą danych, logika biznesowa serwisów oraz endpointy REST API działają zgodnie ze specyfikacją, a warstwa frontendowa renderuje komponenty w przewidziany sposób.

Architektura projektu obejmuje dwa główne moduły:

- **Backend** — serwis Spring Boot (Java 21) eksponujący REST API, komunikujący się z bazą danych PostgreSQL, przechowujący pliki w chmurze Cloudflare R2 (kompatybilnej z AWS S3) oraz integrujący się z serwisem Google OAuth.
- **Frontend** — aplikacja mobilna napisana w React Native z Expo, skierowana na platformy Android i iOS.

Niniejszy raport opisuje strategię testowania, zastosowane narzędzia, wyniki pokrycia kodu, a także szczegółowe omówienie każdego rodzaju testów.

2 Narzędzia testowe

2.1 Backend

Na backendzie zastosowano: **JUnit 5** jako framework do pisania i uruchamiania testów, **Mockito** do tworzenia atrap zależności w testach jednostkowych, **Testcontainers** do uruchamiania rzeczywistej bazy PostgreSQL na potrzeby testów integracyjnych, **JaCoCo** do generowania raportów pokrycia kodu oraz **Postman** do manualnego testowania REST API.

2.2 Frontend

Na frontendzie zastosowano: **Jest** jako framework testowy, **jest-expo** jako preset konfiguracyjny dla projektów Expo oraz **@testing-library/react-native** do renderowania i weryfikacji komponentów React Native w izolowanym środowisku.

3 Pipeline CI/CD

Projekt posiada skonfigurowany pipeline Continuous Integration oparty na **GitHub Actions**, zdefiniowany w pliku `.github/workflows/ci.yml`. Pipeline uruchamia się automatycznie przy każdym wypchnięciu zmian na gałąź `main` oraz przy każdym otwartym pull requeście do tej gałęzi.

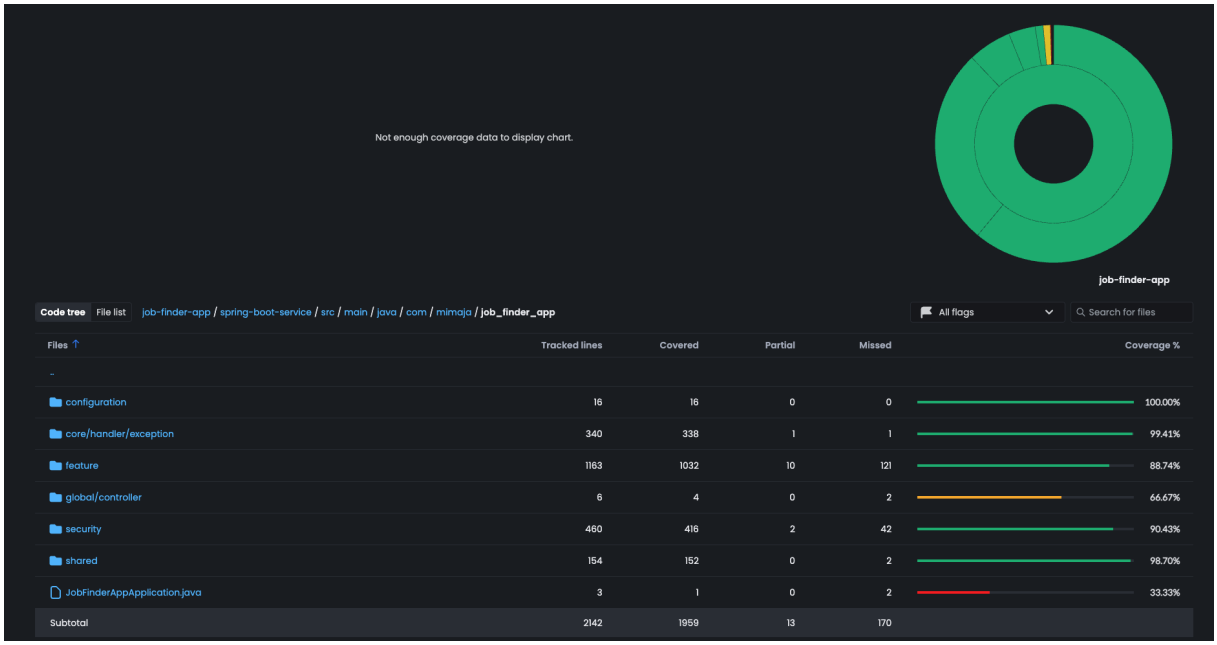
Pipeline składa się z dwóch równoległych zadań (ang. *jobs*):

1. **Mobile Frontend** — zadanie uruchamiane na systemie `ubuntu-latest` z Node.js 20. Instaluje zależności poleceniem `npm install --legacy-peer-deps`, a następnie wykonuje testy z flagami `--ci --coverage`, co wymusza tryb nieinteraktywny oraz generowanie raportu pokrycia.

2. **Spring Boot Service Backend** — zadanie uruchamiane na `ubuntu-latest` z Java 21 w dystrybucji Temurin, korzystające z cache'owania zależności Gradle. Sekwencja kroków obejmuje: nadanie uprawnień do uruchamiania wrappera Gradle, wykonanie zadań `clean test jacocoTestReport jacocoTestCoverageVerification --no-daemon`, wygenerowanie podsumowania JaCoCo w podsumowaniu kroku GitHub Actions (za pomocą skryptu Python), przesłanie raportu XML do platformy **Codecov** oraz archiwizację raportu HTML JaCoCo jako artefaktu GitHub Actions.

Codecov — szczegółowe raporty pokrycia

Narzędzie **Codecov** jest podpięte do każdego uruchomienia pipeline'u backendowego. Pozwala przeglądać szczegółowe raporty JaCoCo z podziałem na katalogi i pliki, śledzić trendy pokrycia w czasie oraz otrzymywać automatyczne komentarze w pull requestach z różnicą pokrycia. Raporty dostępne są pod flagą `backend` i obejmują plik `jacocoTestReport.xml` generowany przez Gradle.



Rysunek 1: Widok pokrycia poszczególnych modułów backendu w narzędziu Codecov

4 Pokrycie kodu

4.1 Backend

Pokrycie kodu backendu mierzone jest przez **JaCoCo** i przesyłane automatycznie do Codecov przy każdym pomyślnym uruchomieniu pipeline'u. Poniższa tabela przedstawia szczegółowe wyniki pokrycia uzyskane z raportu `jacocoTestReport.xml`:

Metryka	Pokryte	Łącznie	Procent
Instrukcje (Instruction)	8 849	10 900	**81,18%**
Gałęzie (Branch)	433	474	**91,35%**
Linie (Line)	2 034	2 453	**82,92%**
Metody (Method)	414	542	**76,38%**
Klasy (Class)	142	181	**78,45%**

Wysokie pokrycie gałęzi na poziomie **prawie 92%** oznacza, że niemal wszystkie ścieżki warunkowe kodu zostały przetestowane. Niższy wynik dla metod i klas wynika z celowego pominięcia w testach warstwy kontrolerów (testy manualne przez Postman) oraz klas konfiguracyjnych infrastruktury (np. konfiguracja bezpieczeństwa Spring Security, enkodery), które z natury są trudne do testowania automatycznego bez uruchomionego kontenera serwletów.

Obszary bez automatycznego pokrycia

Spośród pakietów wyłączonych lub nisko pokrytych automatycznymi testami wymienić należy: `security/encoder` (konfiguracja BCrypt, 0%), kontrolery REST (0% — testowane manualnie przez Postman), `security/smsCode/service` (0% — funkcjonalność w fazie rozwoju), `configuration` (0% — klasy konfiguracji Springa) oraz `feature/user/filterspecification` (3,9%). Są to obszary, w których testy manualne lub integracyjne na poziomie HTTP przejmują rolę automatycznej weryfikacji.

job-finder-app

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
com.mimaja.job_finder_app.security.configuration		0%		0%	14	14	55	55	12	12	2	2
com.mimaja.job_finder_app.feature.job.cotroller		0%		n/a	10	10	28	28	10	10	1	1
com.mimaja.job_finder_app.feature.offer.controller		0%		n/a	5	5	20	20	5	5	1	1
com.mimaja.job_finder_app.security.smsCode.service		0%		0%	5	5	25	25	4	4	1	1
com.mimaja.job_finder_app.security.encoder		0%		n/a	14	14	24	24	14	14	4	4
com.mimaja.job_finder_app.security.shared.utils		0%		n/a	4	4	25	25	4	4	1	1
com.mimaja.job_finder_app.feature.user.filterspecification		3%		0%	8	10	13	15	1	3	0	1
com.mimaja.job_finder_app.feature.cv.controller		0%		n/a	6	6	19	19	6	6	1	1
com.mimaja.job_finder_app.feature.offer.tag.controller		0%		n/a	6	6	18	18	6	6	2	2
com.mimaja.job_finder_app.feature.contract.controller		0%		n/a	7	7	14	14	7	7	1	1
com.mimaja.job_finder_app.feature.offer.tag.category.controller		0%		n/a	5	5	17	17	5	5	2	2
com.mimaja.job_finder_app.feature.user.controller		0%		n/a	4	4	15	15	4	4	1	1
com.mimaja.job_finder_app.feature.application.controller		0%		n/a	5	5	10	10	5	5	1	1
com.mimaja.job_finder_app.feature.user.update.controller		0%		n/a	3	3	9	9	3	3	1	1
com.mimaja.job_finder_app.feature.offer.tag.category.filterspecification		8%		0%	4	6	7	9	1	3	0	1
com.mimaja.job_finder_app.configuration		0%		n/a	2	2	16	16	2	2	2	2
com.mimaja.job_finder_app.shared.enums		73%		n/a	1	5	7	26	1	5	1	3
com.mimaja.job_finder_app.security.authorization.googleAuth.controller		0%		n/a	3	3	6	6	3	3	1	1
com.mimaja.job_finder_app.shared.dto		38%		n/a	3	4	5	6	3	4	1	2
com.mimaja.job_finder_app.global.controller		0%		n/a	6	6	6	6	6	6	3	3
com.mimaja.job_finder_app.feature.contract.dto.cloudflare		0%		n/a	2	2	3	3	2	2	1	1
com.mimaja.job_finder_app.feature.contract.model		47%		0%	2	3	8	16	1	2	0	1
com.mimaja.job_finder_app.feature.contract.utils		0%		n/a	1	1	7	7	1	1	1	1
com.mimaja.job_finder_app.feature.job.jobDispatcher.model		68%		0%	2	5	7	21	1	4	0	3
com.mimaja.job_finder_app.feature.user.profilephoto.model		45%		0%	2	3	7	13	1	2	0	1
com.mimaja.job_finder_app.feature.offer.offerphoto.model		45%		0%	2	3	7	13	1	2	0	1
com.mimaja.job_finder_app.feature.job.jobphoto.model		43%		0%	2	3	7	13	1	2	0	1
com.mimaja.job_finder_app.feature.user.update.password.website.controller		0%		n/a	2	2	5	5	2	2	1	1
com.mimaja.job_finder_app.security.smsCode.model		0%		n/a	1	1	5	5	1	1	1	1
com.mimaja.job_finder_app.security.token.accessToken.utils		0%		n/a	3	3	4	4	3	3	1	1
com.mimaja.job_finder_app.security.token.refreshToken.controller		0%		n/a	1	1	3	3	1	1	1	1
com.mimaja.job_finder_app.feature.user.profileCompletionForm.controller		0%		n/a	1	1	2	2	1	1	1	1
com.mimaja.job_finder_app.feature.user.update.password.mobile.controller		0%		n/a	1	1	2	2	1	1	1	1
com.mimaja.job_finder_app.security.authorization.login.controller		0%		n/a	1	1	2	2	1	1	1	1
com.mimaja.job_finder_app.security.authorization.register.controller		0%		n/a	1	1	2	2	1	1	1	1
com.mimaja.job_finder_app.security.token.accessToken.authorizationFilter		96%		100%	3	22	1	75	3	7	0	2
com.mimaja.job_finder_app		0%		n/a	2	2	3	3	2	2	1	1
com.mimaja.job_finder_app.core.handler.exception.controller		0%		n/a	2	2	2	2	2	2	1	1
com.mimaja.job_finder_app.feature.offer.filterspecification		97%		95%	1	13	0	30	0	3	0	1
com.mimaja.job_finder_app.feature.offer.tag.service		97%		100%	0	17	1	31	0	15	0	3
com.mimaja.job_finder_app.feature.offer.tag.category.service		96%		100%	0	15	1	32	0	13	0	3
com.mimaja.job_finder_app.feature.job.model		94%		50%	1	3	0	16	0	2	0	2
com.mimaja.job_finder_app.feature.offer.tag.filterspecification		93%		87%	1	7	0	14	0	3	0	1

(a) Pakiety z niskim lub zerowym pokryciem — kontrolery, konfiguracja, SMS

com.mimaja.job_finder_app.feature.offer.tag.filterspecification		93%		87%	1	7	0	14	0	3	0	1
com.mimaja.job_finder_app.core.handler.exception		99%		96%	2	80	1	338	0	53	0	8
com.mimaja.job_finder_app.feature.job.service		100%		100%	0	64	0	175	0	38	0	2
com.mimaja.job_finder_app.feature.offer.service		100%		100%	0	29	0	92	0	19	0	2
com.mimaja.job_finder_app.feature.contract.service		100%		100%	0	29	0	87	0	12	0	1
com.mimaja.job_finder_app.feature.offer.mapper		100%		100%	0	17	0	96	0	8	0	1
com.mimaja.job_finder_app.feature.application.service		100%		100%	0	25	0	64	0	16	0	2
com.mimaja.job_finder_app.security.authorization.googleAuth.service		100%		100%	0	13	0	76	0	4	0	1
com.mimaja.job_finder_app.shared.utils		100%		100%	0	33	0	55	0	13	0	1
com.mimaja.job_finder_app.feature.cv.service		100%		100%	0	17	0	47	0	16	0	2
com.mimaja.job_finder_app.feature.user.update.service		100%		100%	0	9	0	50	0	4	0	1
com.mimaja.job_finder_app.feature.user.mapper		100%		100%	0	12	0	65	0	6	0	1
com.mimaja.job_finder_app.security.token.refreshToken.service		100%		100%	0	11	0	48	0	8	0	1
com.mimaja.job_finder_app.shared.service		100%		100%	0	16	0	35	0	7	0	1
com.mimaja.job_finder_app.feature.job.mapper		100%		100%	0	9	0	53	0	5	0	1
com.mimaja.job_finder_app.feature.offer.dto		100%		n/a	0	6	0	6	0	6	0	6
com.mimaja.job_finder_app.feature.user.service		100%		n/a	0	12	0	35	0	12	0	3
com.mimaja.job_finder_app.feature.user.dto		100%		n/a	0	6	0	6	0	6	0	6
com.mimaja.job_finder_app.security.token.resetToken.service		100%		100%	0	7	0	30	0	5	0	1
com.mimaja.job_finder_app.shared.adapters		100%		n/a	0	14	0	23	0	14	0	2
com.mimaja.job_finder_app.feature.offer.tag.category.model		100%		n/a	0	2	0	19	0	2	0	2
com.mimaja.job_finder_app.feature.offer.tag.mapper		100%		100%	0	12	0	33	0	6	0	1
com.mimaja.job_finder_app.feature.user.update.password.utils		100%		100%	0	10	0	24	0	4	0	2
com.mimaja.job_finder_app.feature.user.update.password.website.service		100%		n/a	0	3	0	19	0	3	0	1
com.mimaja.job_finder_app.feature.offer.tag.category.mapper		100%		100%	0	7	0	27	0	4	0	1
com.mimaja.job_finder_app.security.token.accessToken.service		100%		100%	0	2	0	18	0	1	0	1
com.mimaja.job_finder_app.feature.cv.dto		100%		n/a	0	5	0	10	0	5	0	3
com.mimaja.job_finder_app.feature.user.model		100%		n/a	0	3	0	19	0	3	0	2
com.mimaja.job_finder_app.feature.application.mapper		100%		100%	0	5	0	20	0	3	0	1
com.mimaja.job_finder_app.feature.user.update.shared.requestDto		100%		n/a	0	6	0	6	0	6	0	6
com.mimaja.job_finder_app.feature.cv.model		100%		100%	0	4	0	19	0	3	0	1
com.mimaja.job_finder_app.feature.offer.model		100%		n/a	0	3	0	15	0	3	0	2
com.mimaja.job_finder_app.feature.cv.mapper		100%		100%	0	5	0	17	0	3	0	1
com.mimaja.job_finder_app.feature.user.profileCompletionForm.service		100%		n/a	0	1	0	14	0	1	0	1
com.mimaja.job_finder_app.security.authorization.register.service		100%		n/a	0	2	0	11	0	2	0	1
com.mimaja.job_finder_app.feature.job.jobDispatcher.dto		100%		n/a	0	4	0	8	0	4	0	2
com.mimaja.job_finder_app.shared.record		100%		n/a	0	2	0	9	0	2	0	1
com.mimaja.job_finder_app.feature.contract.dto.response		100%		100%	0	4	0	7	0	2	0	1
com.mimaja.job_finder_app.feature.user.update.password.mobile.service		100%		100%	0	3	0	11	0	2	0	1
com.mimaja.job_finder_app.security.authorization.login.utils		100%		100%	0	4	0	12	0	1	0	1
com.mimaja.job_finder_app.security.authorization.login.service		100%		100%	0	4	0	10	0	2	0	1
com.mimaja.job_finder_app.feature.offer.tag.category.dto		100%		n/a	0	4	0	4	0	4	0	4
com.mimaja.job_finder_app.feature.offer.offerphoto.dto		100%		n/a	0	3	0	4	0	3	0	2
com.mimaja.job_finder_app.feature.user.profilephoto.dto		100%		n/a	0	3	0	4	0	3	0	2
com.mimaja.job_finder_app.feature.offer.tag.dto		100%		n/a	0	3	0	3	0	3	0	3
com.mimaja.job_finder_app.feature.job.dto		100%		n/a	0	1	0	1	0	1	0	1
com.mimaja.job_finder_app.feature.application.dto		100%		n/a	0	2	0	2	0	2	0	2
com.mimaja.job_finder_app.security.shared.dto		100%		n/a	0	2	0	2	0	2	0	2
com.mimaja.job_finder_app.security.authorization.googleAuth.dto.request		100%		n/a	0	3	0	3	0	3	0	3

(a) Pakiety z wysokim pokryciem — serwisy, mappery, filtry

4.2 Frontend

Pokrycie kodu frontendowego jest trudne do jednoznacznego oszacowania. Poza testami komponentów uruchamianymi przez Jest, aplikacja frontendowa była testowana **manualnie** — co jest standardową praktyką w projektach React Native, gdzie znaczna część logiki jest ściśle związana z natywną warstwą platformy, animacjami i gestami, których automatyczne testowanie jest niepraktyczne lub niemożliwe bez rzeczywistego urządzenia.

5 Zasady pisania testów

Projekt posiada spisane, obowiązujące zasady tworzenia testów backendowych, dostępne w pliku `TESTING_RULES.md` w korzeniu repozytorium. Poniżej przedstawiono skrót najważniejszych reguł.

Reguła 1 — Jedna asercja na test

Każdy test weryfikuje dokładnie **jedną właściwość** lub **jeden efekt uboczny**. Dzięki temu nazwa testu w pełni opisuje sprawdzane zachowanie, a po niepowodzeniu testu natychmiast wiadomo, co poszło nie tak. Jeśli testowane są zarówno wynik metody, jak i interakcja z mockiem, tworzone są dwa osobne testy.

Reguła 2 — Konwencja nazewnictwa

Metody testowe stosują konwencję `nazwaMetody_shouldOczekiwaneZachowanie_whenWarunek`. Segment `_when` jest opcjonalny, gdy warunek wynika jednoznacznie z nazwy, ale obowiązkowy przy wielu wariantach tego samego wywołania. Testy integracyjne stosują natomiast konwencję BDD: `shouldReturnResult_whenCondition`.

Reguła 3 — Stałe `private static final`

Wszystkie literały łańcuchowe i liczbowe używane w testach są wyciągane do pól `private static final`. Eliminuje to tzw. *magic values*, ułatwia zmianę wartości w jednym miejscu i nadaje im znaczącą nazwę. Stałe współdzielone między wieloma klasami testowymi umieszczane są w dedykowanych klasach danych testowych (pakiet `mockdata`).

Reguła 4 — `@BeforeEach` tylko dla danych wspólnych

W metodzie `setUp()` umieszczane są wyłącznie dane potrzebne zdecydowanej większości lub wszystkim testom w klasie. Zmienne używane tylko w kilku testach inicjalizowane są lokalnie lub w prywatnych metodach pomocniczych. Metoda `setUp()` musi być opatrzona adnotacją `@BeforeEach` — nigdy nie wywołujemy jej ręcznie.

Reguła 5 — Ekstrakcja konfiguracji mocków do metod pomocniczych

Gdy kilka testów wymaga identycznej konfiguracji mocków niepasującej do `@BeforeEach`, konfigurację tę wyciągamy do prywatnej metody pomocniczej. Dotyczy to zarówno konfiguracji `when/thenReturn`, jak i samego tworzenia obiektów przez `mock()`.

Reguła 6 — Struktura Given / When / Then

Każdy test integracyjny jest pisany w strukturze trzech faz, oddzielonych komentarzami `// given`, `// when`, `// then`. Zapewnia to jednolitość i czytelność kodu testowego.

Reguła 7 — Klasy danych testowych (MockData)

Dane testowe nie są tworzone bezpośrednio w testach. Zamiast tego korzysta się z dedykowanych klas fabrycznych w pakiecie `mockdata` (`UserMockData`, `JobMockData`, `SecurityMockData` itd.). Metody i stałe z tych klas importowane są przez `static import`.

Reguła 8 — AssertJ jako preferowana biblioteka asercji

Preferowane są asercje z biblioteki **AssertJ** (`assertThat(...).isEqualTo(...)`) ze względu na czytelniejsze komunikaty błędów i możliwość łańcuchowania. JUnit 5 jest dopuszczalny, ale nie należy mieszać obu stylów w obrębie jednego testu.

Reguła 9 — Testy jednostkowe z Mockito

Testy jednostkowe są adnotowane `@ExtendWith(MockitoExtension.class)`. Mocki wstrzykiwane są przez `@Mock` i `@InjectMocks`. Styl konfiguracji mocków (`when/thenReturn` lub BDD `given/willReturn`) musi być spójny w obrębie jednej klasy testowej.

Reguła 10 — Testy integracyjne z klasami bazowymi

Testy integracyjne dziedziczą z odpowiedniej klasy bazowej (np. `UserIntegrationTest`), która sama dziedziczy z `IntegrationTest`. Klasa bazowa dostarcza kontener bazy danych przez Testcontainers, konfigurację bezpieczeństwa i metody pomocnicze. W `@BeforeEach` zawsze wywoływane jest `super.setUp()`.

Reguła 11 — Testy parametryzowane

Gdy ten sam scenariusz dotyczy wielu wartości wejściowych, stosowany jest `@ParameterizedTest` z `@MethodSource`. Dostawcy danych umieszczani są w klasach MockData lub jako statyczne metody w klasie testowej.

Reguła 12 — Kolejność metod w klasie testowej

Przyjęta kolejność metod w klasie testowej to: (1) pola klasy, (2) metoda `@BeforeEach` `setUp()`, (3) metody testowe, (4) prywatne metody pomocnicze — zawsze na końcu pliku.

6 Podsumowanie liczby testów

Poniższa tabela przedstawia szczegółowy rozkład testów backendowych według rodzaju i obszaru domenowego. Łączna liczba testów backendowych wynosi **941**, w tym **857 testów jednostkowych** i **84 testy integracyjne**.

Rodzaj testu	Obszar domenowy	Liczba testów
Jednostkowy	Bezpieczeństwo i autoryzacja (logowanie, rejestracja, Google Auth, tokeny, filtry)	125
Jednostkowy	Użytkownicy (serwisy, mapper, aktualizacja, hasła, formularz profilu)	173
Jednostkowy	Oferty i ogłoszenia (serwisy, mapper, filtrowanie)	115
Jednostkowy	Tagi i kategorie (serwisy, mappery, filtrowanie)	101
Jednostkowy	Prace (serwisy, mapper)	117
Jednostkowy	CV (serwisy, mapper, model)	55
Jednostkowy	Aplikacje na oferty (serwisy, mapper)	49
Jednostkowy	Kontrakty (serwis)	43
Jednostkowy	Współdzielone (walidacja danych, zarządzanie plikami, adaptery)	79
Suma — testy jednostkowe		**857**
Integracyjny	Autoryzacja i bezpieczeństwo (logowanie, rejestracja, Google, chronione trasy)	12
Integracyjny	Użytkownik (profil, hasła, panel admina)	15
Integracyjny	Oferty, tagi i kategorie	17
Integracyjny	Praca — Job	8
Integracyjny	CV	6
Integracyjny	Aplikacje na oferty	6
Integracyjny	Kontrakty	6
Integracyjny	Globalne i międzymodułowe	14
Suma — testy integracyjne		**84**
ŁĄCZNIE — BACKEND		**941**

Testy frontendowe obejmują **21 plików testowych** z łącznie **48 przypadkami testowymi** (ang. *test cases*) pokrywającymi komponenty interfejsu użytkownika.

7 Rodzaje testów

7.1 Frontend

7.1.1 Testy komponentów

Testy komponentów frontendowych pisane są przy użyciu **Jest** oraz **@testing-library/react-native**. Każdy test renderuje komponent w izolowanym środowisku i weryfikuje obecność oczekiwanych elementów interfejsu — zgodnie z filozofią testowania z perspektywy użytkownika, a nie szczegółów implementacyjnych. Dla wybranych komponentów generowane są snapshoty, które pozwalają wykryć niezamierzone zmiany w strukturze renderowanego drzewa.

7.1.2 Testy manualne

Aplikacja frontendowa była intensywnie testowana **manualnie** — co jest powszechną praktyką w ekosystemie React Native, gdzie wiele aspektów działania (animacje gestów, integracja z kamerą, przejścia między ekranami) jest trudnych do pokrycia automatycznie bez rzeczywistego urządzenia. Manualne testy obejmowały wszystkie kluczowe przepływy użytkownika, takie jak logowanie, przeglądanie ofert gestem swipe, zarządzanie CV czy składanie aplikacji.

7.2 Backend

7.2.1 Testy integracyjne

Testy integracyjne weryfikują poprawność działania całych przepływów żądanie-odpowiedź przez rzeczywiste endpointy HTTP, z uruchomionym kontenerem serwletów i prawdziwą bazą danych PostgreSQL dostarczaną przez **Testcontainers**. Wszystkie klasy testów integracyjnych dziedziczą ze wspólnej klasy bazowej `IntegrationTest`, która dostarcza konfigurację kontenera, ustawienia bezpieczeństwa oraz metody pomocnicze (m.in. tworzenie użytkowników testowych i pobieranie tokenów). Szczególną rolę pełni klasa `FeatureModulesIntegrationTest`, która weryfikuje kompatybilność modułów poprzez złożone przepływy łączące kilka endpointów jednocześnie — np. rejestrację użytkownika, dodanie oferty, złożenie aplikacji i podpisanie kontraktu.

7.2.2 Testy jednostkowe

Testy jednostkowe stanowią najliczniejszą grupę testów backendowych. Testują izolowaną logikę biznesową serwisów, mapperów i komponentów pomocniczych — bez uruchamiania kontenera Spring ani bazy danych — przy użyciu **Mockito** do zastępowania zależności zewnętrznych atrapami.

7.2.3 Testy manualne

Warstwa REST API backendu była testowana manualnie przy użyciu **Postman**. Kolekcja żądań obejmuje znaczną większość endpointów aplikacji, pogrupowane według modułów funkcjonalnych,

i umożliwia weryfikację pełnych przepływów biznesowych, kodów statusu HTTP oraz zachowania API przy błędnych danych wejściowych.

Rysunek 4: Struktura kolekcji Postman używanej do manualnego testowania REST API

8 Wnioski

Przeprowadzone testy potwierdzają wysoki poziom jakości kodu backendowego aplikacji Fuchacz. Pokrycie gałęzi na poziomie **prawie 92%** świadczy o tym, że niemal wszystkie ścieżki logiczne zostały zweryfikowane przez testy automatyczne. Łącznie **941 testów backendowych** (857 jednostkowych + 84 integracyjnych) zapewnia szeroką siateczkę bezpieczeństwa chroniącą przed regresją przy dalszym rozwoju projektu.

Mocne strony strategii testowania

Spójne zasady pisania testów (udokumentowane w `TESTING_RULES.md`) zapewniają jednolitość i czytelność kodu testowego. Automatyzacja przez GitHub Actions gwarantuje uruchamianie testów przy każdej zmianie w repozytorium. Integracja z Codecov umożliwia śledzenie trendów pokrycia w czasie. Testcontainers zapewnia wiarygodność testów integracyjnych przez użycie prawdziwej bazy danych PostgreSQL.

Obszary do poprawy

Warstwa kontrolerów REST pozostaje poza zasięgiem testów automatycznych i jest testowana wyłącznie manualnie przez Postman. W przyszłości warto rozważyć rozszerzenie testów integracyjnych o pełniejsze pokrycie scenariuszy błędów (np. nieprawidłowe dane wejściowe, brak uprawnień) na poziomie HTTP. Frontend wymaga ustalenia strategii pomiaru pokrycia — obecny setup generuje raport pokrycia przez Jest, jednak interpretacja wyników jest utrudniona ze względu na specyfikę środowiska React Native.